11. Consumer Setup - Part1

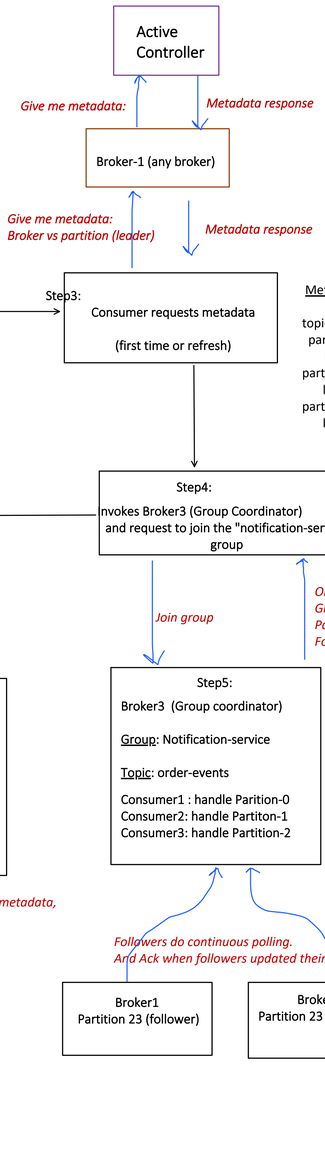
Saturday, 14 March 2026 12:11 PM

In Kafka Architecture topic, I have already covered the overall Consumer workflow:





Complete Consumer Read Flow:



Consumer process events

Consumer commits offset (manual batch wise)

Continuous Polling and move to step 8

Commit

ACK

Step12:

Broker3 (Group Coordinator)

Writes to:
_consumer_offsets, Partition23:
(group, topic, partition)
offset processed till = 501

Step1: Dependency

pom.xml

```
<dependency>
      <groupId>org.springframework.kafka</groupId>
      <artifactId>spring-kafka</artifactId>
</dependency>
```

Step2: application.properties

```
server.port=8082
spring.application.name=kafka-consumer-service

spring.kafka.bootstrap-servers=localhost:9092,localhost:9192

spring.kafka.consumer.group-id=order-consumer-group

spring.kafka.consumer.key-deserializer=org.apache.kafka.common.serialization.StringDeserializer
spring.kafka.consumer.value-deserializer=org.springframework.kafka.support.serializer.JsonDeserializer

spring.kafka.consumer.properties.spring.json.value.default.type=com.eda.consumer.model.Order
```

Which group consumer needs to join

Should be same what at Producer we have selected

Since we have selected JSON Deserializer , we need to tell the default type, in which this JSON need to be converted to.

POJO class

```
public class Order {
    private String orderId;
    private String customerId;
    private String productId;
    private Integer quantity;
    private Double totalAmount;
    private String status;

        //getters and setters
}
```

Step3: Which topic need to listen and Business logic

Topic consumer interested in

```
@Component
public class OrderEventListener {

@KafkaListener(topics="order-events")
public void consume (Order order) {
      //business logic
      System.out.println("Received order event: " + order.getOrderId());
   }
}
```

For each record(event), this method will get executed

DEMO

Now lets go 1 level deeper, what is happening behind the scene:

conceptually what we have to do without spring boot is:

```
Properties props = new Properties();
    props.put("bootstrap.servers", "localhost:9092,localhost:9192");
    props.put("group.id", "order-consumer-group");
    props.put("key.deserializer", StringDeserializer.class);
    props.put("value.deserializer", JsonDeserializer.class);
    props.put("auto.offset.reset", "latest");
    props.put("enable.auto.commit", "true");

KafkaConsumer consumer = new KafkaConsumer<>(props);
consumer.subscribe(Arrays.asList("order-events"));

Connect with 1 of the broker and fetch cluster metadata and store it

try {
        while (true) {
            ConsumerRecords records = consumer.poll()
                Internally poll() method, If it's a First call:
                    • Sends JoinGroupRequest to Group Coordinator
                    • Receives partition assignment
                    • Request last committed offset for assigned partition from  group coordinator
                    • Group coordinator reads from _consumer_offsets and return the offset that need to be read.
                    • Sends FetchRequest to partition leaders with offset which it want to read

                If not a first calls:
                    •  For subsequent calls, Sends FetchRequest for new data (new offsets in continuous manner)

            //commitIntervalPassed default is 5sec, after every 5 second, make offset commit request
             if (autoCommitEnabled && commitIntervalPassed) {
                commitOffsets();
```

Which group consumer needs to join

Should be same what producer has selected

Which topic, this consumer is interested in.

Continuous poll inside While(true) loop, and it will fetch the records

If auto-commit is enabled, then after specific time interval, it commit the offset.

No matter if fetch poll records are successfully processed or not.

```
            }

        for (ConsumerRecord record : records) {        <-- Process the events
                //business logic
            }

        If enable.auto.commit=false, we must call:        --> If auto-commit is false, then Manually send the commit request
            • consumer.commitSync(); // blocks until broker confirms
            • consumer.commitAsync(); // fire-and-forget, no guarantee
        }
    }
    finally {
        consumer.close(); // leaves consumer group
        no heartbeat send by consumer background thread to broker, broker consider it dead and triggers rebalance
    }
}
```

Now good thing is, we don't have to write all of this by ourself, its all provided within Spring Kafka framework.

We have to provide this

```
Properties props = new Properties();
    props.put("bootstrap.servers", "localhost:9092,localhost:9192");
    props.put("group.id", "order-consumer-group");
    props.put("key.deserializer", StringDeserializer.class);
    props.put("value.deserializer", JsonDeserializer.class);
    props.put("auto.offset.reset", "latest");
    props.put("enable.auto.commit", "true");
```

→ application.properties

```
spring.kafka.bootstrap-servers=localhost:9092,localhost:9192
spring.kafka.consumer.group-id=order-consumer-group
spring.kafka.consumer.key-deserializer=org.apache.kafka.common.serialization.StringDeserializer
spring.kafka.consumer.value-deserializer=org.springframework.kafka.support.serializer.JsonDeserializer
spring.kafka.consumer.properties.spring.json.value.default.type=com.eda.consumer.model.Order
spring.kafka.consumer.auto-offset-reset=latest //default
spring.kafka.consumer.enable-auto-commit=true //default
```

```
KafkaConsumer consumer = new KafkaConsumer<>(props);
```

→ ConsumerFactory used to creates an object of KafkaConsumer using application.properties

ConsumerFactory

DeafultKafkaConsumerFactory

Framework provided this

```
consumer.subscribe(Arrays.asList("order-events"));
```

```
Connect with 1 of the broker and fetch cluster metadata and store it

try {
        while (true) {
            ConsumerRecords records = consumer.poll()

            //commitIntervalPassed default is 5sec, after every 5 second, make offset commit request
            if (autoCommitEnabled && commitIntervalPassed) {
                commitOffsets();
            }

        for (ConsumerRecord record : records) {
                //business logic
            }
```

KafkaListenerContainer

@KafkaListener

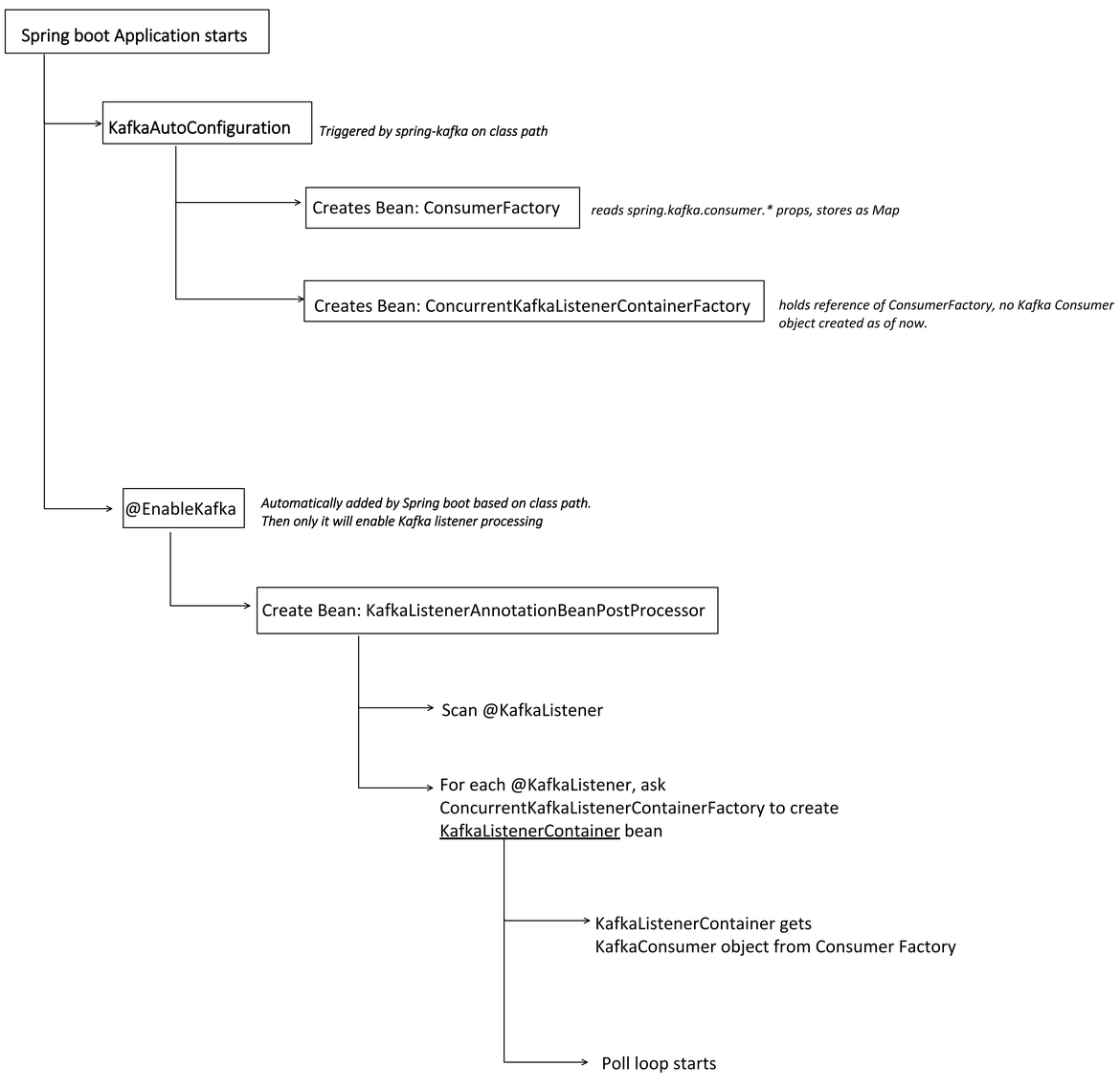
```
@KafkaListener(topics = "order-events")
    public void consume(Order order) {
    //business logic
    System.out.println("Received order event: " + order);
}
```

We have to provide this

```
        If enable.auto.commit=false, we must call:
            • consumer.commitSync(); // blocks until broker confirms
            • consumer.commitAsync(); // fire-and-forget, no guarantee
        }
    }
    finally {
        consumer.close(); // leaves consumer group
        no heartbeat send by consumer background thread to broker, broker consider it dead and triggers rebalance
    }
}
```

Framework provided this

Flow is:


Spring boot Application starts
KafkaAutoConfiguration
Triggered by spring-kafka on class path
Creates Bean: ConsumerFactory
reads spring.kafka.consumer.* props, stores as Map
Creates Bean: ConcurrentKafkaListenerContainerFactory
holds reference of ConsumerFactory, no Kafka Consumer object created as of now.
@EnableKafka
Automatically added by Spring boot based on class path.
Then only it will enable Kafka listener processing
Create Bean: KafkaListenerAnnotationBeanPostProcessor
Scan @KafkaListener
For each @KafkaListener, ask ConcurrentKafkaListenerContainerFactory to create KafkaListenerContainer bean
KafkaListenerContainer gets KafkaConsumer object from Consumer Factory
Poll loop starts